



POLITECNICO
MILANO 1863

Scuola di Ingegneria Industriale e dell'Informazione
Master's Degree in High Performance Computing Engineering

Space and Time Adaptivity

Balancing Accuracy and Performance in PDE Solvers

Authors:

Crippa Tommaso, 10776593

Arnaboldi Leonardo, 10773068

Donato Luca, 10660050

Felipe Epia, 11008027

December 23, 2025

Abstract

This project explores adaptive methods in both space and time for solving partial differential equations. We focus on the heat equation with a localized, time-dependent forcing term characterized by sharp, frequent impulses. The goal is to capture steep solution features accurately while minimizing computational cost through dynamic mesh and time step refinement based on error estimates. We implemented an adaptive solver using the `deal.II` library and evaluated its effectiveness in terms of accuracy and efficiency.

Contents

1	Introduction	1
1.1	Related Work	1
2	Mathematical model	2
2.1	Weak formulation	2
2.2	Space Discretization	2
2.3	Time Discretization	4
2.4	Well-posedness and convergence under uniform refinement	4
3	Adaptivity	5
3.1	Uniform refinement	5
3.2	Space adaptivity	5
3.3	Time adaptivity	6
3.4	Combined space-time adaptivity	7
4	Implementation	8
4.1	Parallelization	8
4.1.1	Distributed mesh and degree of freedom management	8
4.1.2	Linear algebra and matrix assembly	8
4.1.3	Parallel mesh refinement and solution transfer	9
4.1.4	Communication and I/O	9
5	Results	10
5.1	Method of Manufactured Solutions (MMS) and forcing function	10
5.2	Accuracy benchmark	10
5.3	Scalability results on Mare Nostrum Supercomputer	12
5.3.1	Strong scalability analysis	13
5.3.2	Weak scalability analysis	14
6	Conclusions	16
6.1	Future work	16

1 Introduction

Solving partial differential equations with high accuracy often requires significant computational resources, especially in regions where the solution exhibits sharp gradients or rapid changes. Adaptive methods in space and time offer a principled way to allocate computational effort selectively. By refining the resolution only where and when necessary, these techniques improve accuracy while keeping the overall computational cost manageable.

Spatial adaptivity allows the numerical solver to concentrate mesh elements in regions with complex spatial features, such as steep gradients or localized phenomena. This targeted refinement leads to a more accurate approximation of the solution without uniformly increasing the number of degrees of freedom. In practical simulations, this can significantly reduce memory usage and solver time compared to uniform meshes.

Temporal adaptivity plays a similar role in time integration. By adjusting the time step based on the evolving behavior of the solution, the solver avoids unnecessary computations during smooth phases and captures moving dynamics more accurately during rapid changes. This results in improved stability and precision, especially for problems with localized, time-dependent forcing terms. Adaptive methods are therefore essential tools for balancing accuracy and efficiency in numerical simulations.

1.1 Related Work

Several recent works have proposed significantly improved strategies for adaptivity in both space and time compared to standard fixed-grid approaches. Notably, methods based on wavelet-in-time refinement [4, 5] and fully unstructured space–time finite elements [2] have demonstrated superior efficiency and accuracy through more flexible and error-sensitive discretizations.

In our implementation, we specifically took inspiration from other techniques: For spatial adaptivity, we followed the approach detailed in the `deal.II` step-6 tutorial [1], which combines classical mesh refinement with error estimation. For time adaptivity, we implemented the method introduced by Picasso [3], which uses a posteriori error estimates to control the timestep selection in a backward Euler scheme. These methods formed the foundation of our adaptive solver.

2 Mathematical model

In this project, we consider the heat equation with homogeneous Neumann boundary conditions and vanishing initial data:

$$\begin{cases} \frac{\partial u}{\partial t} - \nabla \cdot (\mu \nabla u) = f & \text{in } \Omega \times (0, T), \\ \mu \nabla u \cdot \mathbf{n} = 0 & \text{on } \partial\Omega \times (0, T), \\ u = 0 & \text{in } \Omega \times \{0\}, \end{cases}$$

where the spatial domain is the unit cube $\Omega = (0, 1)^3$, $T > 0$ is the final time, and $\mu > 0$ is a constant diffusion coefficient, which in our tests is set to $\mu = 1$. The boundary condition $\mu \nabla u \cdot \mathbf{n} = 0$ prescribes a vanishing normal heat flux on $\partial\Omega$, and therefore corresponds to homogeneous Neumann boundary conditions.

For the sake of brevity, we denote the forcing term simply as $f(x, t)$, postponing the description of the specific functions used in the experiments to **Section 5**. We assume that $f \in L^2(\Omega)$, a condition that ensures the well-posedness of the continuous problem under homogeneous Neumann boundary conditions.

2.1 Weak formulation

Let us introduce the space

$$H^1(\Omega) = \{v \in L^2(\Omega) : \nabla v \in (L^2(\Omega))^3\},$$

and set

$$V = H^1(\Omega).$$

Multiplying the equation by $v \in V$ and integrating over Ω , and using the homogeneous Neumann boundary condition to eliminate the boundary term, we obtain the following weak formulation (PW):

Find $u(t) \in V$ such that

$$\begin{cases} \int_{\Omega} \partial_t u(x, t) v(x) dx + \int_{\Omega} \nabla u(x, t) \cdot \nabla v(x) dx = \int_{\Omega} f(x, t) v(x) dx, & \forall v \in V, t \in (0, T], \\ u(0) = 0. \end{cases} \quad (1)$$

2.2 Space Discretization

We start from the heat equation with homogeneous Neumann boundary conditions and an initial condition, which defines a continuous problem with infinitely many degrees of freedom. In order to obtain a tractable formulation, we discretize and approximate the

weak form in space.

To this aim, we introduce a conforming finite element space in dimension $d = 3$:

$$X_h = \{v_h \in C^0(\bar{\Omega}) : v_h|_{K_i} \in \mathbb{P}_r, \forall K_i \in \mathcal{T}_h\},$$

where $\mathbb{P}_r$ denotes the space of polynomials of degree less than or equal to r on each element of the tetrahedral triangulation $\mathcal{T}_h$ forming an approximation of $\Omega \subset \mathbb{R}^3$. We then define the discrete finite dimensional space for the approximate solution as

$$V_h = X_h \cap V,$$

where $V = H^1(\Omega)$ is the continuous space for the weak formulation.

The semi-discrete problem, obtained through a Galerkin approximation in space, reads as follows:

$$\forall t > 0, \text{ find } u_h(t) \in V_h : \quad \int_{\Omega} \partial_t u_h v_h dx + \int_{\Omega} \nabla u_h \cdot \nabla v_h dx = \int_{\Omega} f v_h dx \quad \forall v_h \in V_h,$$

with homogeneous initial condition

$$u_h(x, 0) = 0.$$

Since $V_h \subset V$, we have that $\dim(V_h) = N_h$. Let $\{\phi_j\}_{j=1}^{N_h}$ be a basis of V_h . Then we represent the semi-discrete approximate solution as

$$u_h(x, t) = \sum_{j=1}^{N_h} U_j(t) \phi_j(x),$$

where $\{U_j(t)\}_{j=1}^{N_h}$ are time-dependent unknown coefficients.

Choosing $v_h = \phi_i$ in the semi-discrete formulation, we obtain the following finite-dimensional system:

$$\sum_{j=1}^{N_h} \dot{U}_j(t) \int_{\Omega} \phi_j \phi_i dx + \sum_{j=1}^{N_h} U_j(t) \int_{\Omega} \nabla \phi_j \cdot \nabla \phi_i dx = \int_{\Omega} f \phi_i dx, \quad i = 1, \dots, N_h,$$

with initial condition

$$U(0) = 0.$$

This leads to a system of ordinary differential equations in time for the vector of unknown coefficients $U(t) \in \mathbb{R}^{N_h}$.

2.3 Time Discretization

The semi-discrete formulation of the heat equation is further discretized in time using the Crank–Nicolson (CN) method. Let $u_h^n \in V_h$ denote the finite element approximation of the solution u at time $t^n = n\Delta t$. We aim to compute u_h^{n+1} at time t^{n+1} .

The time derivative is approximated by

$$\frac{u_h^{n+1} - u_h^n}{\Delta t},$$

while the diffusion operator is treated by the trapezoidal rule, i.e., averaging its contribution at times t^n and t^{n+1} .

The fully discrete Crank–Nicolson scheme reads: find $u_h^{n+1} \in V_h$ such that for all $v_h \in V_h$,

$$\int_{\Omega} \frac{u_h^{n+1} - u_h^n}{\Delta t} v_h dx + \frac{1}{2} \int_{\Omega} \nabla u_h^{n+1} \cdot \nabla v_h dx + \frac{1}{2} \int_{\Omega} \nabla u_h^n \cdot \nabla v_h dx = \frac{1}{2} \int_{\Omega} f^{n+1} v_h dx + \frac{1}{2} \int_{\Omega} f^n v_h dx$$

The problem is spatially discretized using the Finite Element Method and temporally discretized using the Crank–Nicolson scheme. This scheme is second-order accurate in time and unconditionally stable for the heat equation.

2.4 Well-posedness and convergence under uniform refinement

The well-posedness of the weak problem is guaranteed by the properties of the bilinear and linear forms. Indeed, the bilinear form $a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v dx$ is coercive and continuous on $V \times V$, and the linear functional $F(v) = \int_{\Omega} f v dx$ is continuous, provided that $f \in L^2(\Omega)$ by hypothesis. Under these conditions, the problem admits a unique weak solution.

For the spatial discretization, we consider conforming finite elements of degree $r = 2$ on a family of shape-regular meshes $\{\mathcal{T}_h\}$ covering $\Omega \subset \mathbb{R}^3$. Let $V_h \subset V$ denote the associated conforming finite element space. Then the semi-discrete solution u_h satisfies the standard convergence estimates under uniform mesh refinement,

$$\|u(t) - u_h(t)\|_{L^2(\Omega)} \leq Ch^3, \quad \|u(t) - u_h(t)\|_{H^1(\Omega)} \leq Ch^2,$$

for all $t \in [0, T]$, where h denotes the maximum element diameter and $C > 0$ is a constant independent of h .

For the time discretization, we apply a Crank–Nicolson scheme of order $p = 2$ on a uniform partition of $[0, T]$ with time step Δt . Under suitable regularity assumptions on u , the fully discrete solution u_h^n at time $t_n = n\Delta t$ satisfies

$$\|u(t_n) - u_h^n\|_{L^2(\Omega)} \leq C_1 h^2 + C_2 (\Delta t)^2$$

3 Adaptivity

In this work, we explore two strategies for controlling the accuracy of the numerical solution of the heat equation: a uniform refinement strategy and a fully adaptive space-time approach. The main goal of adaptivity is to efficiently allocate computational resources, refining the mesh or reducing the time step only where and when it is necessary, while avoiding unnecessary computations in regions where the solution is smooth.

3.1 Uniform refinement

As a baseline, we first consider uniform refinement of the spatial mesh. In this approach, the computational domain is discretized using a globally uniform grid, and the time step Δt is kept constant throughout the entire simulation. By increasing the number of spatial cells and decreasing the time step uniformly, we can systematically improve the accuracy of the solution, recovering in practice the convergence rates predicted by the a priori error estimates above.

Uniform refinement provides a simple and robust way to study convergence and to generate reference results for comparison. However, it has significant limitations: it does not take into account the local behavior of the solution. In regions where the solution changes slowly, computational resources are wasted, as the fine mesh and small time steps are not strictly necessary. Conversely, areas with steep gradients or localized sources may require even finer resolution than what uniform refinement provides, making this approach inefficient for complex or strongly time-dependent problems.

3.2 Space adaptivity

To overcome the limitations of uniform refinement, we implement an adaptive refinement strategy in space. The idea is to refine the spatial mesh locally according to an a posteriori error indicator, thereby concentrating computational effort where it is most needed while keeping a coarser mesh elsewhere.

In spatial adaptivity, the mesh is dynamically refined or coarsened based on cellwise error indicators. Cells where the estimated error exceeds a given threshold are refined, increasing the local resolution, while cells with sufficiently low error are coarsened to reduce the number of degrees of freedom. The percentages of cells to refine or coarsen are selected to achieve a balance between accuracy and computational efficiency.

To implement space adaptivity inside our heat-equation solver, we adopt the standard "solve-estimate-mark-refine" loop. At each refinement step, the Kelly error estimator identifies cells where the finite element solution u_h exhibits large gradient jumps across cell faces. These jumps correlate with poor local approximation and are quantified by the

error indicator

$$\eta_K^2 = \sum_{F \subset \partial K} h_F \int_F \llbracket \nabla u_h \cdot n \rrbracket^2 d\sigma,$$

where the sum runs over the faces F of the cell K , h_F is a characteristic length of the face, and $\llbracket \nabla u_h \cdot n \rrbracket$ denotes the jump of the normal derivative across F . The global spatial error indicator is then defined as

$$\eta = \left(\sum_K \eta_K^2 \right)^{1/2},$$

which provides an a posteriori estimate of the spatial discretization error.

After computing the indicators η_K , a marking strategy (for instance, marking a fixed fraction of cells with largest η_K) selects cells to be refined or coarsened, and a mesh refinement routine updates the triangulation accordingly.

After computing the indicators η_K , we apply a fixed-fraction marking strategy. In practice, we prescribe two user-defined percentages: a refinement fraction θ_{ref} and a coarsening fraction θ_{coarse} . All cells whose error indicator belongs to the highest θ_{ref} of the distribution (i.e. with the largest values of η_K) are marked for refinement, while cells whose indicator lies within the lowest θ_{coarse} are marked for coarsening. A mesh refinement routine then updates the triangulation accordingly, increasing the resolution where the error is largest and reducing the number of degrees of freedom where the solution is already well resolved.

3.3 Time adaptivity

To complement spatial refinement, we implement time adaptivity to accurately capture fast temporal dynamics while maintaining computational efficiency. Our approach follows the classical idea of local error control through step-doubling.

The time step size Δt is adapted dynamically at selected iterations based on a local error estimate. This estimate is computed by comparing a single full time step of size Δt with two successive half-steps of size $\Delta t/2$. Let $u_{\Delta t}^{(1)}$ be the solution after a single step of size Δt , and $u_{\Delta t/2}^{(2)}$ the solution after two half-steps. The estimated temporal error is then

$$e_t \approx \|u_{\Delta t}^{(1)} - u_{\Delta t/2}^{(2)}\|_{L^2(\Omega)}.$$

If e_t is above a prescribed upper tolerance, the step is rejected and repeated with a smaller time step. If e_t is within the admissible tolerance interval, the step is accepted and the time step is kept unchanged. If e_t is well below a lower tolerance threshold, the step is accepted and the time step is increased for the next iteration, up to a maximum allowed value. This controller allows the solver to take larger time steps in smooth phases and reduces Δt automatically during rapid solution changes, while ensuring that the temporal error remains within user-defined bounds.

3.4 Combined space-time adaptivity

The combination of spatial and temporal adaptivity allows the solver to focus computational resources both where and when they are most needed. Regions of the domain with steep gradients receive finer spatial resolution, and the time steps are adjusted to capture fast dynamics accurately.

In practice, we combine the spatial and temporal strategies described above in a single, but not fully coupled, adaptive loop. Spatial and temporal adaptation are both applied only at selected iterations, in order to limit the overhead. More precisely, spatial adaptivity is triggered every few time steps, when the cellwise error indicators are evaluated and a fixed percentage of cells with the largest indicators is refined, while a smaller fraction with the smallest indicators is coarsened. Temporal adaptivity is activated at prescribed time levels, where a local time-discretization error is estimated and compared against upper and lower tolerances to decide whether to reject the current step, keep the time step unchanged, or increase it for the next iteration.

By dynamically refining the mesh and adjusting the time step only where and when necessary, the adaptive method is designed to concentrate effort in regions with high solution complexity, thus reducing the overall number of degrees of freedom and computational cost.

The main objective of this project is to verify in practice that a space-time adaptive solver can achieve a comparable level of accuracy to the one obtained by uniform refinement, while requiring fewer computational resources. Through numerical experiments, we systematically compare uniform and adaptive strategies, focusing on both error metrics and runtime.

4 Implementation

4.1 Parallelization

The numerical solver is designed to run on distributed-memory systems using MPI. The parallelization strategy employs deal.II's distributed triangulation and Trilinos linear algebra libraries, which provide scalable data structures and algorithms optimized for parallel computing.

4.1.1 Distributed mesh and degree of freedom management

The computational mesh is managed through the use of `parallel::distributed::Triangulation<dim>`, which automatically partitions the domain among MPI processes. Each process owns a subset of cells (the locally owned cells) and stores ghost cells, neighboring cells that belong to other processes but are needed for local computations and communication at subdomain boundaries.

Degrees of freedom are distributed via the `DoFHandler`, which creates an `IndexSet` for each process:

- *Locally owned DoFs*: The degrees of freedom associated with cells owned by a given process.
- *Locally relevant DoFs*: A superset that includes DoFs on ghost cells. These are necessary for assembling local contributions to the system matrices and for computing error estimates.

This two-level indexing structure ensures that assembly loops over local cells can access all needed DoF values without global communication at every operation.

4.1.2 Linear algebra and matrix assembly

System matrices and vectors are stored using Trilinos (through deal.II's `TrilinosWrappers` interface), which provides distributed implementations of sparse matrices and vectors. Specifically:

- Mass matrix, stiffness matrix, and time-stepping matrices (`lhs_matrix`, `rhs_matrix`) are of type `TrilinosWrappers::SparseMatrix`, distributed across processes.
- Solution vectors use `TrilinosWrappers::MPI::Vector`:
 - `solution_owned`: Contains only the locally owned components, used for computations.
 - `solution`: A ghosted vector (includes locally relevant DoFs) used for output and computing error estimates.

Matrix assembly is performed locally: each process assembles contributions only from its owned cells, then all processes collectively compress the matrices via `VectorOperation::add` to accumulate contributions at shared DoFs.

4.1.3 Parallel mesh refinement and solution transfer

Adaptive mesh refinement in parallel is handled via `parallel::distributed::GridRefinement::refine_and_coarsen_fixed_number()`, which marks cells for refinement and coarsening in a load-balanced manner. When the mesh is refined or coarsened:

1. Error indicators are computed locally on each process's cells.
2. A global marking strategy selects which cells to refine or coarsen.
3. The mesh is coarsened and refined, and DoFs are redistributed.
4. `parallel::distributed::SolutionTransfer<dim,TrilinosWrappers::MPI::Vector>` interpolates the solution from the old mesh to the new mesh.
5. All data structures are rebuilt on each process.

This approach ensures that the load is rebalanced after refinement, preventing one process from becoming a bottleneck.

4.1.4 Communication and I/O

MPI communication is minimized by design:

- Vector assembly requires only local sparse accumulations; global communication occurs implicitly in the compressed format.
- Global reductions (e.g., computing the global L^2 norm of a vector) use `Utilities::MPI::` utility functions, which handle efficient collective operations.

5 Results

5.1 Method of Manufactured Solutions (MMS) and forcing function

To rigorously verify the numerical accuracy of our adaptive finite element solver, we employed the Method of Manufactured Solutions (MMS) using a superposition of pulsating Gaussian fields. Within this framework, the forcing function $f(x, t)$ is derived by substituting the chosen manufactured solution into the governing partial differential equation.

Unlike comparing results against experimental data or high-resolution reference simulations ("Golden References"), MMS allows for a pure mathematical verification of the algorithm implementation.

Pulsating Gaussian fields have 2 important properties:

- We can choose when and where to have a spike in the field, this will allow us to test temporal adaptivity.
- We can setup how steep we want the spike to be, so it can be as short and small as we want, this will allow us to test spatial adaptability.

$$\text{Governing Equation: } \frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2} = f(x, t) \quad (2)$$

$$\text{Manufactured Solution: } u(x, t) = \underbrace{\left(\sum_{i=1}^{N_x} a_i e^{-\frac{(x-c_i)^2}{2\sigma_i^2}} \right)}_{X(x)} \underbrace{\left(\sum_{j=1}^{N_t} b_j e^{-\frac{(t-\tau_j)^2}{2\delta_j^2}} \right)}_{T(t)} \quad (3)$$

$$\text{Forcing Function: } f(x, t) = X(x)\mathcal{T}'(t) - \alpha \mathcal{X}''(x)T(t)$$

$$\text{where: } \mathcal{T}'(t) = \sum_{j=1}^{N_t} -\frac{b_j(t-\tau_j)}{\delta_j^2} e^{-\frac{(t-\tau_j)^2}{2\delta_j^2}}$$

$$\text{and: } \mathcal{X}''(x) = \sum_{i=1}^{N_x} a_i \left[\frac{(x-c_i)^2}{\sigma_i^4} - \frac{1}{\sigma_i^2} \right] e^{-\frac{(x-c_i)^2}{2\sigma_i^2}} \quad (4)$$

Although the forcing function is analytically defined, it was computed in practice through the numerical evaluation of the exact solution's derivatives within the solver.

5.2 Accuracy benchmark

We conducted a series of numerical experiments to evaluate the performance and efficiency of the proposed space-time adaptive method in comparison with the traditional uniform

refinement strategy. All simulations were executed on a local workstation with 20 cores using MPI parallelization. Keeping the forcing function consistent in all tests.

The primary objective of the accuracy benchmark is to assess the trade-off between computational cost (measured by wall-clock time) and solution accuracy (quantified by the relative L^2 error at the final time).

Figure 1 visualizes the cost-accuracy trade-off for both methods on a log-log scale.

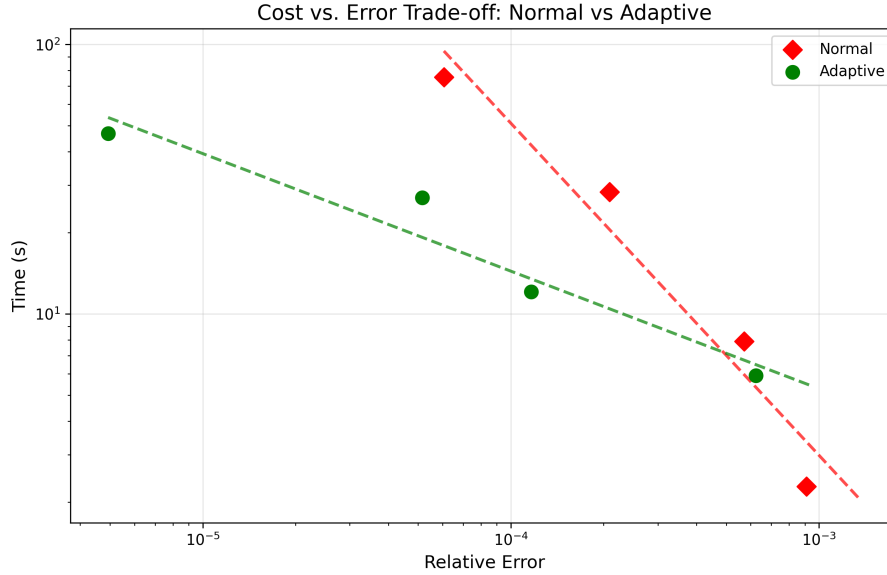
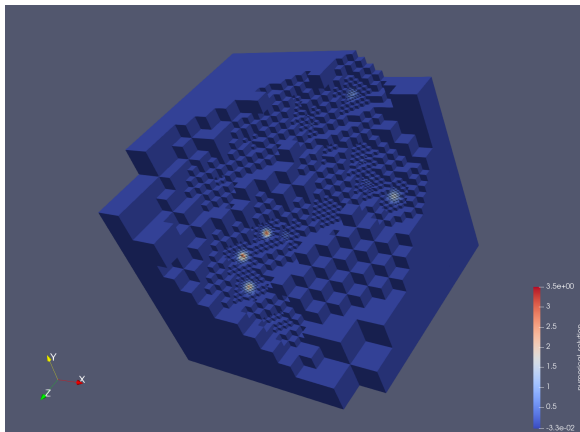


Figure 1: Cost vs. error trade-off for uniform (red) and adaptive (green) refinement strategies.

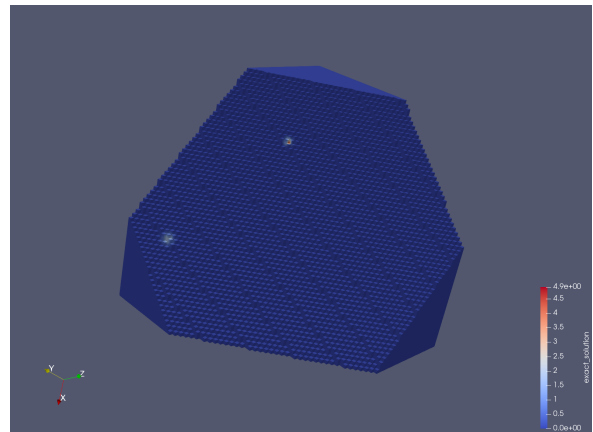
The plot reveals several key insights:

- **Superior accuracy at equivalent cost:** The adaptive method consistently achieves significantly lower errors than uniform refinement for comparable computational times. This demonstrates that by concentrating computational resources in regions with high error indicators, the adaptive strategy uses the available degrees of freedom far more efficiently.
- **Faster convergence for high-accuracy targets:** When targeting very low error levels, the adaptive method shows even more dramatic advantages. The finest adaptive simulation achieves an error of 4.93×10^{-6} in 46.75 seconds, whereas the finest uniform refinement only reaches 6.06×10^{-5} in 75.58 seconds, requiring more than 60% additional time while still producing an error more than ten times larger.
- **Efficiency at coarse resolution:** When introducing coarser refinement levels, the uniform method starts to become better, as it achieves similar errors with less time taken. This overhead in the adaptive case is attributed to the cost of error estimation and mesh refinement operations.

- **Convergence rates:** The approximately linear trends in the log-log plot indicate power-law relationships between computational cost and error. The adaptive method exhibits a steeper slope, implying a faster rate of error reduction per unit of computational effort. This is consistent with theoretical predictions that adaptive refinement can recover optimal convergence rates while using significantly fewer degrees of freedom than uniform approaches.



(a) Adaptive



(b) Uniform

These results confirm that the space-time adaptive solver successfully addresses the primary goal of the project: achieving comparable or superior accuracy to uniform refinement while substantially reducing computational cost, particularly for problems with sharp, localized features.

5.3 Scalability results on Mare Nostrum Supercomputer

To evaluate the performance and scalability of our solver, we conducted a series of tests on the MareNostrum 5 supercomputer of the Barcelona Supercomputing Center. For this purpose, we adopted a container-based approach using Singularity. The container encapsulates the full application together with all required dependencies, allowing us to submit standard Slurm jobs while seamlessly interfacing with the MPI runtime provided by the system. This setup simplifies execution, output collection, and logging, while ensuring a consistent runtime environment across different nodes of the cluster.

To guarantee reproducibility and avoid issues related to binary compatibility, we compiled the entire software toolchain from source inside the container. This includes the compiler toolchain, all external libraries required by the solver, as well as the `deal.II` and Trilinos libraries. By controlling the build configuration and library versions explicitly, we ensure that numerical results and performance measurements are not affected by differences in the underlying system environment.

5.3.1 Strong scalability analysis

To evaluate the parallel performance of the solver, we performed a strong scaling test on the MareNostrum general-purpose partition. The benchmarks were conducted using an exact solution featuring 128 spatial peaks and 5 temporal peaks, with an initial global refinement level of 6. The jobs were submitted to the `gp_debug` queue via Slurm scripts, varying the number of CPU cores from 4 up to 112, which represents the maximum number of cores available on a single node of the GP partition.

The results of the scaling test are summarized in Table 1 and visualized in the performance plot of Figure 3.

CPUUs	Wall Time [s]	Ideal Time [s]
4	512.69	512.69
8	266.70	256.35
16	143.26	128.17
32	79.35	64.09
64	49.74	32.04
112	40.30	18.31

Table 1: Strong scaling timing results on MareNostrum (GP partition).

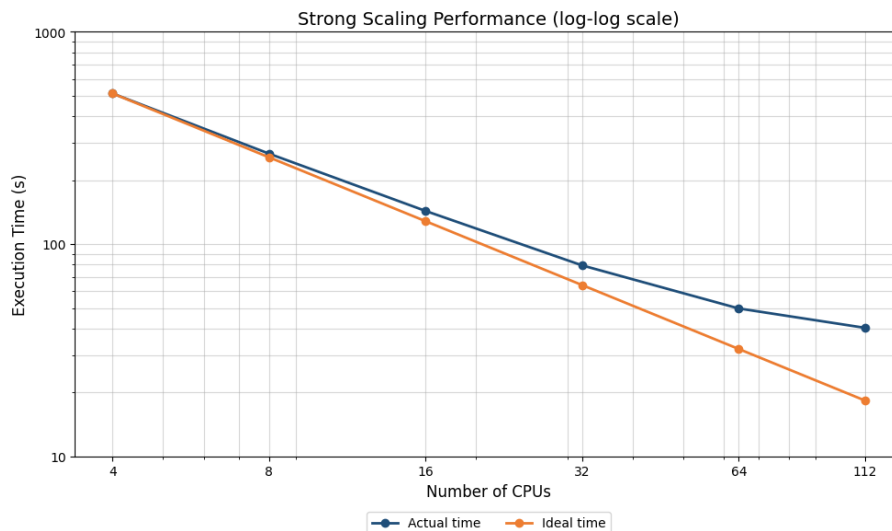


Figure 3: Strong scaling: execution time vs. ideal linear scaling (log-log scale).

As shown in the data, the solver demonstrates good scalability up to approximately 32 cores, where the execution time closely follows the ideal linear speedup. However, as the number of processors increases further, the performance curve begins to deviate from the ideal trend.

This behavior can be attributed to several factors:

- **Communication and Synchronization:** As the number of processors increases, the overhead associated with MPI communication and global synchronization grows, eventually accounting for a larger fraction of the total wall time.

- **Granularity:** The base simulation is relatively short, taking approximately 500 seconds on 4 cores. At higher core counts (e.g., 112 cores), the local problem size per processor becomes too small to provide sufficient computation to effectively hide the parallel overhead.
- **Amdahl's Law:** The presence of even small non-parallelizable sections of the code limits the maximum achievable speedup as the parallel execution time decreases.

5.3.2 Weak scalability analysis

In addition to the strong scaling test, we conducted a weak scaling analysis to observe how the solver handles an increasing problem size alongside proportional computational resources. For this study, the problem size was defined by the number of Degrees of Freedom (DoFs) at the end of the simulation, which was scaled by adjusting the `global refinements` parameter.

We increased the number of CPU cores from 2 up to 64, aiming to maintain a constant workload per processor. The experimental results, summarized in Table 2 and visualized in Figure 4, should be considered indicative, as they highlight the challenges in achieving a linear relationship between the final number of DoFs and the total computational effort.

Global Refinements	DoFs (N)	CPUs	Wall Time [s]	Ideal Time [s]	Efficiency
4	120,375	2	30.44	30.44	1.00
5	619,019	12	37.21	30.44	0.82
6	3,470,000	64	49.89	30.44	0.61

Table 2: Weak scaling results on MareNostrum (GP partition).

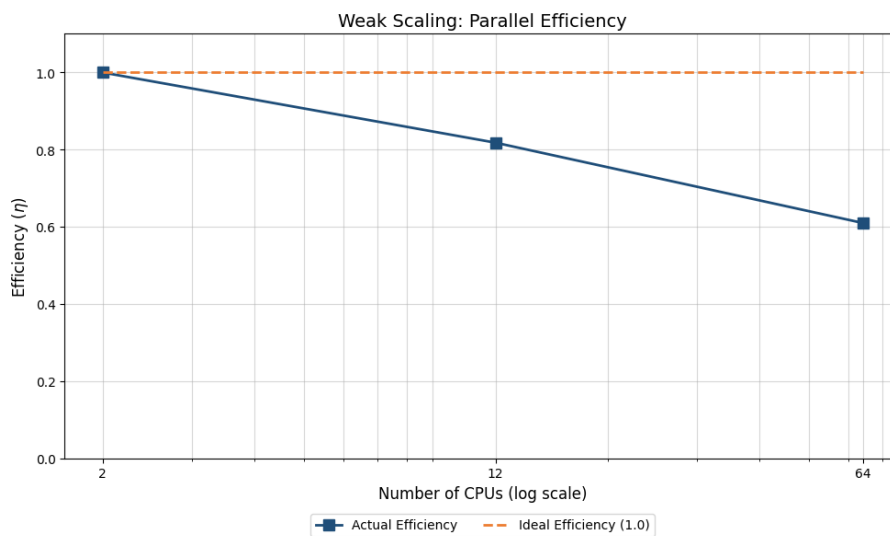


Figure 4: Parallel efficiency for the weak scaling test case.

The data show a drop in parallel efficiency to 0.61 at the highest core count. This departure from the ideal constant execution time is likely due to a combination of factors, such as:

- **Super-linear Complexity:** The computational complexity does not scale strictly as $O(N)$. As the mesh is refined, the conditioning of the linear system typically degrades, requiring more iterations to reach convergence. Furthermore, the final DoF count is a static snapshot that may not fully capture the cumulative cost of adaptive temporal iterations.
- **Parallel and Hardware Overhead:** As observed in the strong scaling test, increasing the number of CPU cores introduces higher MPI communication and synchronization overhead. Additionally, using a large number of cores on a single node leads to memory bandwidth bottlenecks, where multiple processes compete for shared hardware resources, further impacting performance.

In conclusion, while the solver handles increased problem sizes, the observed efficiency loss suggests that larger-scale simulations would benefit from further optimization of communication patterns to mitigate hardware-level constraints.

6 Conclusions

In this project, we investigated the effectiveness of space and time adaptivity for the numerical solution of parabolic partial differential equations, focusing on the heat equation with localized and highly time-dependent forcing terms. The main objective was to assess whether adaptive strategies can provide a better balance between accuracy and computational cost compared to classical uniform refinement approaches.

The numerical results demonstrate that space–time adaptivity is particularly effective when the solution exhibits localized spatial features and rapid temporal dynamics. For medium to high accuracy targets, the adaptive strategy consistently achieves lower errors at a reduced computational cost compared to uniform refinement. This confirms that concentrating computational effort in critical space–time regions leads to a more efficient use of degrees of freedom and improved performance.

At the same time, the experiments highlight some intrinsic limitations of adaptive methods. For coarse accuracy requirements, the overhead introduced by error estimation, mesh refinement, and solution transfer can reduce the overall efficiency, making uniform refinement competitive or even preferable in this regime.

Overall, the results validate the central hypothesis of this work: space–time adaptive methods can match or outperform uniform refinement in terms of accuracy while significantly reducing computational cost for problems with localized and transient features. The combination of adaptive algorithms with parallel finite element infrastructures proves to be a robust and flexible approach for large-scale PDE simulations.

6.1 Future work

Several extensions of the present work could further improve both the accuracy and the efficiency of the proposed solver. A natural direction is the introduction of "hp-adaptivity", where the local polynomial degree is adapted in addition to mesh refinement, allowing for a more flexible resolution of smooth and highly localized solution features. Another important aspect concerns the design of more advanced cost models to guide adaptive decisions, taking into account not only error indicators but also computational cost and RAM usage. Finally, a greater set of benchmarks on the integration between space and time adaptivity with different parameters and functions, would provide deeper insight into the performance limits of adaptive space–time methods.

References

- [1] Wolfgang Bangerth, Timo Heister, and Guido Kanschat. The `deal.II` library: The step-6 tutorial program. https://www.dealii.org/current/doxygen/deal.II/step_6.html, 2023.
- [2] Ulrich Langer and Christoph Schafelner. Adaptive space–time finite element methods for non-autonomous parabolic problems with distributional sources. *arXiv preprint arXiv:2003.09248*, 2020.
- [3] Marco Picasso. Adaptive finite elements for a linear parabolic problem. *Computer Methods in Applied Mechanics and Engineering*, 167(3-4):223–237, 1998.
- [4] Rob Stevenson, Jeroen Westerdiep, and Paul van Venetië. A wavelet-in-time, finite element-in-space adaptive method for parabolic evolution equations. *arXiv preprint arXiv:2101.03956*, 2021.
- [5] Paul van Venetië and Jeroen Westerdiep. Efficient space–time adaptivity for parabolic evolution equations using wavelets in time and finite elements in space. *arXiv preprint arXiv:2104.08143*, 2021.